[87] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45, Online, October 2020. Association for Computational Linguistics. URL `https://www.aclweb.org/anthology/2020.emnlp-demos.6`.

[88] David P Woodruff. Optimal space lower bounds for all frequency moments. In *SODA*, volume 4, pages 167–175. Citeseer, 2004.

[89] Felix Wu, Angela Fan, Alexei Baevski, Yann N Dauphin, and Michael Auli. Pay less attention with lightweight and dynamic convolutions. In *The International Conference on Learning Representations (ICLR)*, 2019.

[90] Yunyang Xiong, Zhanpeng Zeng, Rudrasis Chakraborty, Mingxing Tan, Glenn Fung, Yin Li, and Vikas Singh. Nyströmformer: A nystöm-based algorithm for approximating self-attention. In *Proceedings of the AAAI Conference on Artificial Intelligence. AAAI Conference on Artificial Intelligence*, volume 35, page 14138, 2021.

[91] Li Yuan, Yunpeng Chen, Tao Wang, Weihao Yu, Yujun Shi, Zi-Hang Jiang, Francis EH Tay, Jiashi Feng, and Shuicheng Yan. Tokens-to-token vit: Training vision transformers from scratch on imagenet. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 558–567, 2021.

[92] Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, et al. Big bird: Transformers for longer sequences. *Advances in Neural Information Processing Systems*, 33, 2020.

[93] Shuangfei Zhai, Walter Talbott, Nitish Srivastava, Chen Huang, Hanlin Goh, Ruixiang Zhang, and Josh Susskind. An attention free transformer. *arXiv preprint arXiv:2105.14103*, 2021.

[94] Chen Zhu, Wei Ping, Chaowei Xiao, Mohammad Shoeybi, Tom Goldstein, Anima Anandkumar, and Bryan Catanzaro. Long-short transformer: Efficient transformers for language and vision. *Advances in Neural Information Processing Systems*, 34, 2021.

A Related Work

IO-Aware Runtime Optimization. The broad concept of optimizing for reading and writing to fast/slow memory has a long history in computer science and has been known by many names. We draw the most direct connection to the literature of analyzing I/O complexity in this work [1], but concepts of memory hierarchies are fundamental and has appeared in many forms, from the working set model [21], to data locality [86], to the Roofline model of arithmetic intensity [85], to analyses of scalability [59], to standard textbook treatments of computer architecture [40]. We hope that this work encourages the community to adopt these ideas in more parts of the deep learning stack.

Efficient ML Models with Structured Matrices. Matrix multiply is the core computational bottleneck of most machine learning models. To reduce the computational complexity, there have been numerous approaches to learn over a more efficient set of matrices. These matrices are called *structured matrices*, which have subquadratic ($o(n^2)$ for dimension $n \times n$) number of parameters and runtime. Most common examples of structured matrices are sparse and low-rank matrices, along with fast transforms commonly encountered in signal processing (Fourier, Chebyshev, sine/cosine, orthogonal polynomials). There have been several more general classes of structured matrices proposed in machine learning: Toeplitz-like [78], low-displacement rank [49], quasi-separable [25]). The butterfly pattern we use for our block-sparse attention is motivated by the fact that butterfly matrices [15, 64] and their products have been shown to be able to express any structured matrices with almost optimal runtime and number of parameters [16, 20]. However, even though structured matrices are efficient in theory, they have not seen wide adoption since it is hard to translate their efficiency to wall-clock speedup since dense unconstrained matrix multiply has very optimize implementation, a phenomenon known as the hardware lottery [41]. Extensions of butterfly matrices [17, 18] aimed to make butterfly matrices more hardware-friendly.

Sparse Training. Our block-sparse FlashAttention can be seen as a step towards making sparse model training more efficient. Sparse models have seen success in compressing models for inference (pruning) by sparsifying the weight matrices [23, 38, 39, 55, 76]. For model training, the lottery tickets hypothesis [28, 29, 30] suggests that there are a set of small sub-networks derived from a larger dense network that performs as well as the original dense network. Out block-sparse FlashAttention can also be seen as a fixed lottery ticket in the context of attention: we fix the sparsity pattern to be the butterfly pattern through training, and observe that it performs almost as well as the (dense) FlashAttention on the Long-range Arena tasks.

Efficient Transformer. Transformer-based models have become the most widely-used architecture in natural language processing [22] and computer vision [24, 91]. However, one of their computational bottlenecks is that their time and memory scales quadratic in the sequence length. There are numerous approaches to overcome this bottleneck, including approximation with hashing (i.e., sparse) such as Reformer [51] and Smyrf [19] and with low-rank approximation such as Performer [12, 54]. One can even combine sparse and low-rank approximation for better accuracy (e.g., Longformer [3], BigBird [92], Scatterbrain [9], Long-short transformer [94], Combiner [73]). Other approaches include compressing along the sequence dimension to attend to multiple tokens at once [52, 57, 79, 89]. One can also attend over the states from previous sequences to help lengthen the context (e.g., Transformer-XL [14] and Compressive Transformer [69]). We recommend the survey [81] for more details.

There are several lines of work on developing other modules instead of attention to model longer context. HiPPO [35] and its extensions, most notably S4 [31, 36, 37] projects the history on a polynomial basis, allowing accurate reconstruction of the history through state-space models. They combine the strengths of CNNs (efficient training), RNNs (efficient inference), and continuous models (robust to change in sampling rates). LambdaNetworks [2], AFT [93] and FLASH [42] are other attempts at replacing attention in the context of image classification and language modeling.

B Algorithm Details

We first derive the forward and backward passes of attention and show that they can be computed in a memory-efficient manner (requiring extra memory linear instead of quadratic in the sequence length). Though they reduce the amount of extra memory required, naively they still incur quadratic HBM accesses, resulting in slower execution speed. We describe the FlashAttention algorithm to implement both the forward

and the backward passes on GPUs that reduces HBM accesses, leading to both faster runtime and smaller memory footprint.

B.1 Memory-efficient forward pass

The main challenge in making attention memory-efficient is the softmax that couples the columns of $\mathbf{K}$ (and columns of $\mathbf{V}$). Our approach is to compute the softmax normalization constant separately to decouple the columns. This technique [60] has been used in the literature [51, 66] to show that attention computation does not need quadratic *extra* memory (though the number of HBM accesses is still quadratic, resulting in slow run-time).

For simplicity, we omit here the max-shifting step during softmax. The full algorithm in Appendix B.3 contains all the steps.

Recall that given input sequences $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$, we want to compute the attention output $\mathbf{O} \in \mathbb{R}^{N \times d}$:

$$\mathbf{S} = \mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{N \times N}, \quad \mathbf{P} = \mathrm{softmax}(\mathbf{S}) \in \mathbb{R}^{N \times N}, \quad \mathbf{O} = \mathbf{P}\mathbf{V} \in \mathbb{R}^{N \times d}.$$

We have that $S_{ij} = q_i^T k_j$ where q_i and k_j are the i-th and j-th columns of $\mathbf{Q}$ and $\mathbf{K}$ respectively. Define the normalization constants of softmax:

$$L_i = \sum_j e^{q_i^T k_j}. \tag{1}$$

Let v_j be the j-th column of $\mathbf{V}$, then the i-th columns of the output is

$$o_i = P_{i:}\mathbf{V} = \sum_j P_{ij} v_j = \sum_j \frac{e^{q_i^T k_j}}{L_i} v_j. \tag{2}$$

We see that once L_i is computed, we can compute o_i without extra memory by repeatedly summing $\frac{e^{q_i^T k_j}}{L_i} v_j$. Therefore the forward pass can be computed with $O(n)$ extra memory:

1. Compute L_i for all i according to Eq. (1), which takes $O(n)$ extra memory.
2. Compute o_i for all i according to Eq. (2), which takes $O(d)$ extra memory.

B.2 Memory-efficient backward pass

We derive the backward pass of attention and show that it can also be computed with linear memory. Rabe and Staats [66] suggests that the backward pass can be done without quadratic extra memory by applying gradient checkpointing to the memory-efficient forward pass. We instead derive the backward pass explicitly and show how it can be computed in a memory-efficient manner.

Suppose that there is a scalar loss function ϕ, and let the output gradient be $\mathbf{dO} \in \mathbb{R}^{n \times d}$ (where $\mathbf{dO}$ denotes $\frac{\partial \phi}{\partial \mathbf{O}}$). We want to compute the input gradients $\mathbf{dQ}, \mathbf{dK}, \mathbf{dV} \in \mathbb{R}^{n \times d}$ (where $\mathbf{dQ}, \mathbf{dK}, \mathbf{dV}$ denote $\frac{\partial \phi}{\partial \mathbf{Q}}, \frac{\partial \phi}{\partial \mathbf{K}}, \frac{\partial \phi}{\partial \mathbf{V}}$ respectively).

The gradient $\mathbf{dV}$ is easy to see. Applying reverse-mode autodiff by hand (aka the chain rule), we obtain (in matrix notation) $\mathbf{dV} = \mathbf{P}^T \mathbf{dO}$. Thus:

$$dv_j = \sum_i P_{ij} do_i = \sum_i \frac{e^{q_i^T k_j}}{L_i} do_i. \tag{3}$$

Since we already computed L_i, dv_j can be computed without extra memory by repeated summing.

The gradients $\mathbf{dQ}$ and $\mathbf{dK}$ are a little more complicated. We go through the gradients $\mathbf{dP}$ and $\mathbf{dS}$ first. From Eq. (2), we have that $\mathbf{dP} = \mathbf{dO}\mathbf{V}^T$, and so:

$$dP_{ij} = do_i^T v_j.$$

Recall that $P_{i:} = \mathrm{softmax}(S_{i:})$. Using the fact that the Jacobian of $y = \mathrm{softmax}(x)$ is $\mathrm{diag}(y) - yy^T$, we have that

$$dS_{i:} = (\mathrm{diag}(P_{i:}) - P_{i:}P_{i:}^T) dP_{i:} = P_{i:} \circ dP_{i:} - (P_{i:}^T dP_{i:}) P_{i:},$$